

Relatório de Auditoria de Segurança

SharedAuth

1 de setembro de 2026

Escopo auditado	Repositório SharedAuth na íntegra (v0.10.0): os 12 módulos de <code>`sharedauth/`</code> , os dois pacotes de recursos (<code>`messages/`</code> , <code>`ui/`</code>), os templates e o JavaScript embarcados, <code>`pyproject.toml`</code> , o workflow de CI e o histórico completo do Git.
Linguagem	Python 3.13+
Tipo de artefato	Biblioteca (não é aplicação: sem rotas próprias, sem banco, sem contêiner)
Framework	Núcleo sem dependências; integrações opcionais com Flask 3.1+, Flask-WTF, Flask-Limiter e Werkzeug pelo extra <code>`[flask]`</code>
ORM / query builder	Nenhum — a biblioteca não fala com banco (só monta a URL de conexão)
Mecanismo de auth	Sessão de cookie assinado do Flask + Flask-Login no consumidor; CSRF por Flask-WTF; hash de senha por Werkzeug; portão padrão-nega por <code>`before_request`</code>
Frontend	CSS e JavaScript puros (<code>`ui/estatico/`</code>) e dois parciais Jinja de mensagens
Deploy	Nenhum — não há Dockerfile, Compose ou Helm. Só <code>`.github/workflows/ci.yml`</code> e <code>`.github/dependabot.yml`</code>
Consumidores	ConfortoTermico, MegaSena, ControleRendaVariavel (extra <code>`[flask]`</code>) e ControleBancario (só o núcleo)

Nota metodológica

SharedAuth não é uma aplicação: não tem rota, banco, tela nem arquivo de implantação. Isso não torna as cinco categorias inaplicáveis — torna-as aplicáveis ao CONTRATO que a biblioteca oferece aos quatro aplicativos que a consomem. A pergunta auditada não é 'esta rota vaza dado?', e sim 'este contrato permite que o consumidor construa uma rota que vaza dado sem que nada acuse?'. Onde a categoria realmente não tem equivalente, isso está dito de forma explícita na seção 2.3, sem forçar achado.

Categoria pedida	Como foi mapeada nesta stack
1. Banco sem tranca (isolamento de inquilino/dono)	Não há banco nem consulta na biblioteca. O que existe é o contrato do módulo <code>`access`</code> , que decide QUAIS rotas do consumidor exigem sessão. Auditado como: o portão padrão-nega falha fechado? Uma rota nova nasce protegida? Foi conferido também que a biblioteca deliberadamente NÃO decide papéis ou escopo de dado — isso fica com cada aplicativo.

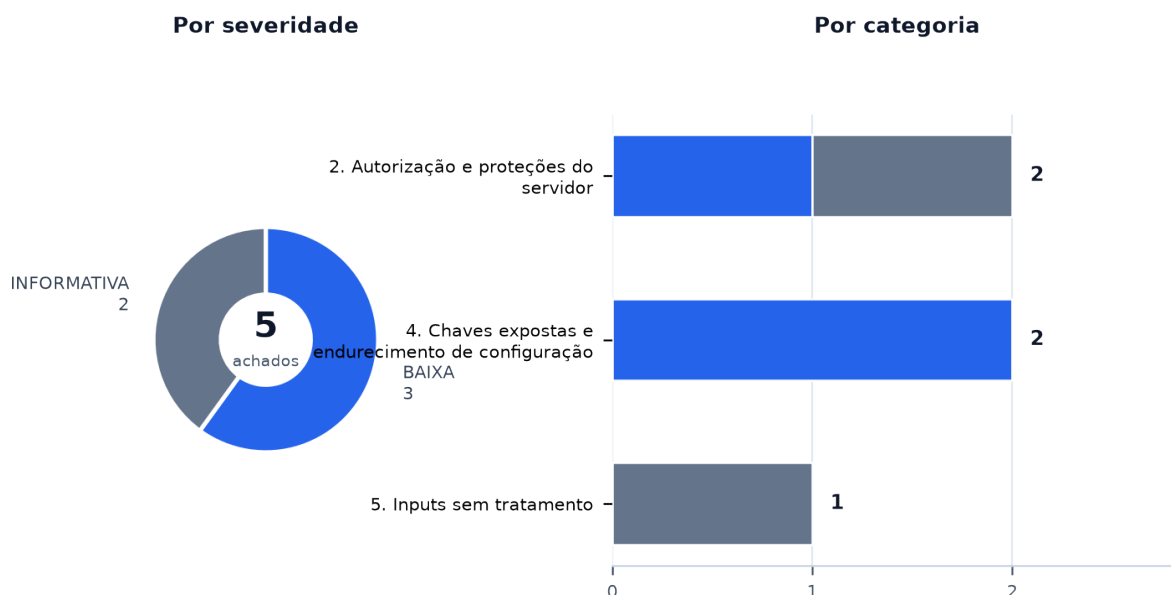
Categoria pedida	Como foi mapeada nesta stack
2. Permissão definida no navegador	Cruzamento do que a biblioteca oferece para o servidor (<code>`requer_papel`</code> , <code>`requer_login`</code> , <code>`requer_troca_de_senha`</code>) com o que ela oferece para a tela (parciais de mensagem, <code>`ui/estatico`</code>). Auditado se algum controle de acesso depende de estado que só existe no cliente.
3. IDOR	Não há rota nem objeto por ID. O equivalente auditado é o único identificador que a biblioteca manipula: o par <code>`id:marca`</code> do cookie de sessão (<code>`session.identificador_de_sessao`</code> / <code>`separar_identificador`</code>) e a comparação da marca de senha.
4. Chaves expostas (hardcode)	Varredura do código, dos testes, do <code>`pyproject.toml`</code> , do workflow de CI e do histórico completo do Git por segredo embutido; e leitura do contrato de <code>`sharedauth.secrets`</code> , que é o mecanismo pelo qual os quatro consumidores obtêm segredo.
5. Inputs sem tratamento (XSS)	Frontend: <code>`ui/estatico/sharedauth-ui.js`</code> (modal e toast montados no navegador) e os dois parciais Jinja de <code>`messages/`</code> . Backend: não há resposta HTML gerada em Python além desses parciais, e não há e-mail nem template renderizado pela biblioteca.

1. Resumo executivo

SharedAuth foi auditada linha por linha e não apresenta nenhuma falha crítica, alta ou média. É a peça mais bem construída do conjunto: o portão de acesso falha fechado, a leitura de segredo nunca coloca o valor numa mensagem de erro, o componente de interface monta todo o DOM por `createElement`/`textContent` (zero `innerHTML` com dado de terceiro) e o histórico do Git não guarda segredo nenhum. Os cinco achados abaixo são de severidade baixa ou informativa e têm um traço em comum: são omissões do CONTRATO, não defeitos de implementação. A biblioteca acerta o que faz; o que ela deixa de fazer é impedir que o consumidor erre — e o efeito disso já é observável nos aplicativos irmãos (ver SA-02 e SA-04, que reaparecem como achados concretos nos relatórios de MegaSena, ControleRendaVariavel e ControleBancario).



Achados por severidade e por categoria



2. Pontos fortes e pontos fracos

2.1 O que está protegido (com evidência)

Controle verificado	Evidência no código
Portão de acesso padrão-nega, com registro único garantido <code>`requer_login`</code> lista os endpoints PÚBLICOS, não os protegidos — uma rota acrescentada amanhã nasce fechada. E o registro duplicado levanta <code>`RuntimeError`</code> em vez de deixar a primeira chamada silenciosamente vencedora, que seria a falha mais difícil de enxergar num controle de acesso.	sharedauth/access.py:58-84
Troca de senha obrigatória verificada em TODA requisição <code>`requer_troca_de_senha`</code> roda como <code>`before_request`</code> , não só no login, e isenta automaticamente o próprio endpoint de troca. Aplicar o desvio apenas no instante do login deixaria a marca ligada e sem efeito: bastaria digitar outra URL para seguir com a senha que o administrador conhece.	sharedauth/access.py:143-163
<code>`url_proximo_seguro`</code> fecha o redirecionamento aberto de forma completa Decodifica em laço (com teto que termina sob aninhamento adversarial), recusa a cada volta o que não começa com <code>`/`</code> , o <code>`//`</code> , a barra invertida e todo caractere de controle, e ainda exige que a forma totalmente decodificada continue sendo caminho. Devolve o valor ORIGINAL, não o decodificado — decodificar mudaria o significado do caminho.	sharedauth/access.py:202-228
Sessão amarrada à senha em vigor, com comparação em tempo constante A marca é HMAC-SHA256 da senha guardada com a <code>`SECRET_KEY`</code> (não um hash simples: não se deriva de um vazamento só do banco), e <code>`marcas_conferem`</code> usa <code>`hmac.compare_digest`</code> . Marca ausente devolve <code>`False`</code> — recusa é o lado seguro.	sharedauth/session.py:96-134
Nenhuma exceção de segredo carrega o valor do segredo Todas as mensagens de <code>`SegredoInvalidoError`</code> nomeiam a variável e o que fazer, nunca o conteúdo. É exigência escrita do módulo, e a leitura linha a linha confirma que ela é cumprida nos quatro caminhos de erro.	sharedauth/secrets.py:62-108
<code>`caminho_esperado`</code> impede que <code>`NOME_FILE`</code> vire seletor arbitrário de arquivo Sem ele, quem controlasse o ambiente do processo apontaria <code>`NOME_FILE`</code> para qualquer arquivo legível e o conteúdo entraria como se fosse o segredo. A comparação é feita sobre caminhos resolvidos dos dois lados, então symlink e <code>`..`</code> não contornam.	sharedauth/secrets.py:80-91
<code>`montar_url_postgres`</code> escapa cada componente com <code>`quote(..., safe=``)`</code></b> Uma senha contendo <code>`@`</code> partiria a URL no lugar errado e a conexão iria para um host que não é o pretendido; uma contendo <code>`/`</code> ou <code>`:`</code> produziria banco ou porta inventados. Falhas que não se anunciam como erro de escape.	sharedauth/config.py:139-142
CSP fechada por padrão, sem <code>`unsafe-inline`</code> em nenhuma diretiva <code>`script-src`</code> e <code>`style-src`</code> fecham em <code>`self`</code> ; <code>`object-src`</code> none; <code>`frame-ancestors`</code> none; <code>`base-uri`</code> self; <code>`form-action`</code> self. A única folga (<code>`img-src data:`</code>) é opt-in por parâmetro e nenhum dos quatro consumidores a usa hoje.	sharedauth/security.py:53-81
Componente de interface sem <code>`innerHTML`</code> com dado de terceiro Modal e toast são montados por <code>`document.createElement`</code> e o texto entra por <code>`textContent`</code> ; os ícones vêm de <code>`createElementNS`</code> . Varredura completa do arquivo (437 linhas) não encontrou <code>`innerHTML`</code> , <code>`outerHTML`</code> , <code>`insertAdjacentHTML`</code> , <code>`eval`</code> , <code>`new Function`</code> nem <code>`document.write`</code> .	sharedauth/ui/estatico/sharedauth-ui.js:102-309

Controle verificado	Evidência no código
Parciais Jinja com autoescape e severidade em lista fechada A mensagem sai por <code>{{ mensagem }}</code> (autoescape do Jinja ligado) e a categoria é reduzida a uma das quatro severidades conhecidas antes de virar classe CSS — uma categoria inventada cai em <code>`info`</code> em vez de virar atributo arbitrário.	sharedauth/messages/templates/sharedauth/flash_messages.html:17-21
Sem segredo no histórico do Git Varredura de todos os commits por arquivo <code>.env`</code> , <code>.pem`</code> , <code>.key`</code> e por padrões de atribuição de segredo em conteúdo: nada. O CI obtém o PAT de leitura por <code>`secrets.SHAREDAUTH_READ_TOKEN`</code> do GitHub, nunca de arquivo versionado.	git log --all (11 commits, v0.1.0..v0.10.0)
<code>`aplicar_limite` / `isentar_limite` religam `view_functions`</code> <code>`RouteLimit.__call__`</code> do Flask-Limiter devolve uma função NOVA; descartar o retorno deixa o limite decorado e nunca aplicado, sem erro e sem aviso. A biblioteca centraliza essa mecânica justamente porque o erro foi cometido três vezes, em três aplicativos diferentes.	sharedauth/ratelimit.py:115-138

2.2 Os riscos centrais

- A biblioteca decide muito bem o que faz e quase nada sobre o que o consumidor deixa de fazer. Os quatro achados de contrato (SA-01, SA-02, SA-04, SA-05) são todos da forma "a biblioteca oferece o mecanismo correto, mas não tem como recusar o uso incorreto" — e cada um deles tem, hoje, um achado concreto correspondente em pelo menos um dos quatro aplicativos consumidores.
- O ponto mais consequente é SA-04: ``resolver_segredo`` aceita a variável direta por padrão e não olha o VALOR. É por essa porta que um segredo de desenvolvimento, guardado em texto claro num `.env`` de máquina local, entraria como configuração operacional sem que nada acusasse. Nos três irmãos isso não chegou a virar exposição — os arquivos estavam fora do Git e fora da sincronização de nuvem —, mas a ausência da trava é o que faz depender de sorte em vez de contrato.
- SA-02 é o segundo: o limitador chaveia por ``get_remote_address``, e atrás de um proxy sem ``ProxyFix`` isso vira um balde único somando o mundo inteiro. Os quatro consumidores hoje resolvem isso corretamente por conta própria, mas a biblioteca não tem como saber disso nem como avisar quando alguém esquecer.
- O piso de senha é de 8 caracteres, sem verificação contra senha comum ou vazada (SA-03). Para o modelo de uso destes aplicativos — poucas contas, todas provisionadas por um administrador — o risco prático é baixo, mas o piso é o mínimo defensável, não uma margem.

2.3 Categorias que não se aplicam a esta stack

- **1. Banco sem tranca (RLS / filtro por tenant)** — SharedAuth não tem banco de dados, não emite consulta SQL e não conhece nenhum modelo de dado. A única função que toca PostgreSQL é ``montar_url_postgres``, que monta a string de conexão e nada mais. O isolamento de dado é decisão de cada aplicativo consumidor e foi auditado nos relatórios deles.
- **3. IDOR (objeto por ID em rota)** — A biblioteca não registra nenhuma rota que receba ID de objeto. ``sharedauth.health.registrar_health`` cria a única rota do pacote (``GET /health``), que não aceita parâmetro e responde apenas ``{servico, status}`` — sem versão, sem nome de banco, sem contagem de registro. O único identificador manipulado é o par ``id:marca`` do cookie, coberto pelos pontos fortes.
- **5. XSS no backend (e-mail/template)** — A biblioteca não envia e-mail, não renderiza template próprio além dos dois parciais de mensagem (auditados e corretos) e não gera resposta HTML em nenhum outro ponto.

3. Achados detalhados

Um bloco por categoria. Todo achado abaixo foi conferido no código real: arquivo, linha e trecho vêm da árvore auditada, não de inferência.

2. Autorização e proteções do servidor

Severidade	Arquivo:linha	Descrição
BAIXA	sharedauth/ratelimit.py:55-62	[SA-02] O limitador chaveia por <code>`get_remote_address`</code> e a biblioteca não tem como saber se há proxy à frente <code>`iniciar_limiter`</code> fixa <code>`key_func=get_remote_address`</code>, que lê <code>`REMOTE_ADDR`</code>. Atrás de um proxy reverso sem <code>`ProxyFix`</code> (ou com o servidor WSGI apagando os cabeçalhos <code>`X-Forwarded-*`</code>), <code>`REMOTE_ADDR`</code> é o endereço do gateway — igual para todo mundo. Por que é explorável: Com a chave colapsada num único valor, o <code>`default_limits`</code> deixa de ser "por cliente" e vira um balde único: quem tenta força bruta divide o orçamento com todos os usuários legítimos, e num aplicativo que faz polling isso é autobloqueio esperando a hora. Não é hipótese — é exatamente o que aconteceu no ConfortoTermico, e o comentário que corrige o problema está escrito em <code>`app_factory.py:309-336`</code> daquele projeto. Impacto: Onde o consumidor esquecer o <code>`ProxyFix`</code>, a proteção de força bruta do login existe no código e não existe em produção, sem nenhum sintoma até alguém ir medir. Condição de explorabilidade: Depende inteiramente do consumidor. Os quatro aplicativos auditados hoje resolvem corretamente (variável de ambiente + <code>`ProxyFix`</code>, e no ConfortoTermico também a configuração de <code>`trusted_proxy`</code> do waitress). O achado é sobre o contrato não oferecer nem <code>`key_func`</code> configurável nem aviso. <pre> limiter = Limiter(key_func=get_remote_address, default_limits=list(limites_padrao) if limites_padrao is not None else None, ...) </pre>

Severidade	Arquivo:linha	Descrição
INFORMATIVA	sharedauth/passwords.py:26 e 52-56	[SA-03] Política de senha é só o piso de 8 caracteres, sem lista de senhas comuns `MIN_PASSWORD_LENGTH = 8` e `validar_tamanho` é a única regra aplicada por `gerar_hash`. Não há verificação contra senha comum, contra semelhança com o nome de usuário nem contra lista de senhas vazadas. Por que é explorável: `senha123` e `12345678` passam. Combinado com o limite de 10 tentativas por minuto por IP (`LIMITE_LOGIN_PADRAO`), um adivinhador com lista curta ainda tem várias centenas de tentativas por hora contra uma conta conhecida. Impacto: Baixo no modelo de uso real: são poucas contas, todas provisionadas por um administrador, e a senha inicial é sempre a temporária gerada por `gerar_senha_temporaria` — 12 caracteres num alfabeto de 58, cerca de 70 bits de entropia, sorteados com `secrets.choice`. O risco fica na senha que o dono escolhe DEPOIS da troca obrigatória. Condição de explorabilidade: Exige uma conta cujo dono tenha escolhido senha fraca e um adivinhador que conheça o nome de usuário. O ControleBancario mitiga por conta própria, com bloqueio persistente por tentativa e o `CommonPasswordValidator` do Django. <pre> MIN_PASSWORD_LENGTH = 8 def validar_tamanho(senha: str) -> None: if len(senha) < MIN_PASSWORD_LENGTH: raise SenhaMuitoCurtaError(...) </pre>

4. Chaves expostas e endurecimento de configuração

Severidade	Arquivo:linha	Descrição
BAIXA	sharedauth/security.py :41-50	[SA-01] `SECURITY_HEADERS` não inclui `Strict-Transport-Security` O conjunto defensivo compartilhado traz seis cabeçalhos (`X-Content-Type-Options`, `X-Frame-Options`, `Referrer-Policy`, `Permissions-Policy`, `Cross-Origin-Opener-Policy`, `X-Permitted-Cross-Domain-Policies`) e nenhum HSTS. `registrar_cabecalhos` também não o acrescenta. Por que é explorável: Nos três consumidores Flask o HSTS existe somente na camada do Nginx (verificado em `deploy/nginx/megasena.conf:26`). Um consumidor publicado sem esse proxy à frente — ou com um proxy reconfigurado sem a diretiva — perde o HSTS por inteiro, e nada na aplicação acusa a ausência. O quarto consumidor (ControleBancario, Django) não depende disso: define `SECURE_HSTS_SECONDS` por conta própria quando `USE_HTTPS=True`. Impacto: Sem HSTS, a primeira requisição de um navegador que ainda não visitou o site pode ser feita em HTTP claro e ser interceptada (downgrade / SSL-strip) antes do redirecionamento para HTTPS. O cookie de sessão só não viaja porque `SESSION_COOKIE_SECURE` está ligado — a proteção depende inteiramente de esse outro interruptor estar correto (ver SA-04). Condição de explorabilidade: Só é explorável quando o aplicativo é servido por HTTPS sem um proxy que já emita HSTS, e contra um navegador na primeira visita (ou com o cache de HSTS expirado). Nas quatro implantações atuais o Nginx cobre o caso. <pre> SECURITY_HEADERS: dict[str, str] = { "X-Content-Type-Options": "nosniff", "X-Frame-Options": "DENY", "Referrer-Policy": "same-origin", "Permissions-Policy": (...), "Cross-Origin-Opener-Policy": "same-origin", "X-Permitted-Cross-Domain-Policies": "none", } </pre>

Severidade	Arquivo:linha	Descrição
BAIXA	sharedauth/secrets.py:93-141	[SA-04] `resolver_segredo` aceita a variável direta por padrão e nunca olha o valor O padrão é `aceitar_variavel=True`: na ausência de `NOME_FILE`, qualquer valor não vazio em `NOME` é aceito como segredo. A função valida a PROCEDÊNCIA (arquivo existe, não está vazio, resolve para o caminho esperado) e nunca o CONTEÚDO — não há gancho para recusar um valor de exemplo, um placeholder de documentação ou um segredo compartilhado com o ambiente de desenvolvimento. Por que é explorável: É por essa porta que um segredo de máquina local pode virar configuração operacional sem nada acusar. Os relatórios irmãos registram a forma branda do efeito: `MegaSena/.env.docker`, `ControleRendaVariavel/.env` e `ControleBancario/.env.docker` guardam senha de banco e chave de sessão em texto claro, duplicando o que já está em `.secrets/`. Nenhum deles vazou — os três estavam fora do Git e fora da sincronização de nuvem, verificado —, e é justamente por isso que o achado é de contrato: nada na biblioteca impediria o contrário, porque nada olha o valor. Impacto: A biblioteca não cria o problema, mas é o único ponto onde ele poderia ser barrado de uma vez para os quatro consumidores. Como está, cada aplicativo teria de escrever a mesma validação por conta própria — que é justamente o padrão que este pacote existe para evitar. Condição de explorabilidade: Não é explorável por si só: é a ausência de uma trava. Torna-se relevante em conjunto com um segredo fraco, de exemplo ou reaproveitado do ambiente local. <pre>if aceitar_variavel: direto = valores.get(nome) if direto: return direto</pre>

5. Inputs sem tratamento

Severidade	Arquivo:linha	Descrição
INFORMATIVA	sharedauth/logs.py:73-122	[SA-05] `sanitizar_log` é rede por reconhecimento de padrão, e o próprio módulo diz isso A redação de credencial usa três expressões regulares (atribuição `chave=valor`, esquema `Bearer`/`Basic`, senha embutida em URL de conexão) sobre uma lista fechada de 12 nomes sensíveis. Um segredo que apareça sem nome reconhecível ao lado atravessa a função sem ser redigido. Por que é explorável: A neutralização de quebra de linha e de caractere de controle — que é a parte que impede a forja de uma linha de log inteira — essa sim é completa e não depende de reconhecimento. O limite está só na redação de credencial. Impacto: Um segredo com nome fora da lista (por exemplo `chave_de_assinatura=` ou `pat=`) chegaria ao log em texto claro. Registrado como informativo porque o módulo declara essa limitação no próprio docstring e aponta a defesa primária correta: não pôr segredo em mensagem, que é o que `sharedauth.secrets` já garante. Condição de explorabilidade: Exige que algum consumidor registre em log um valor sensível cujo rótulo esteja fora da lista. <pre>CHAVES_SENSIVEIS: frozenset[str] = frozenset({ "senha", "password", "passwd", "secret", "token", "authorization", "api_key", "apikey", "access_key", "private_key", "credential", "credentials", })</pre>

4. Recomendações priorizadas

Prio	Ação	Achados
P1	Acrescentar a <code>`resolver_segredo`</code> um parâmetro de recusa de valor (por exemplo <code>`valores_recusados: frozenset[str]`</code> e/ou um piso de comprimento), para que os quatro consumidores possam rejeitar no boot um segredo de exemplo ou reaproveitado de desenvolvimento. É a única correção deste relatório que fecha, de uma vez, uma lacuna presente em três dos quatro aplicativos irmãos.	SA-04
P2	Publicar <code>`Strict-Transport-Security`</code> a partir da própria biblioteca, atrás de um parâmetro explícito de <code>`registrar_cabecalhos`</code> (por exemplo <code>`hsts_segundos: int   None = None`</code>), alimentado pela mesma flag de ambiente que o consumidor já usa para decidir <code>`SESSION_COOKIE_SECURE`</code> . Assim a garantia deixa de depender de a camada do Nginx estar correta em cada implantação.	SA-01
P2	Permitir que <code>`iniciar_limiter`</code> receba <code>`key_func`</code> , e documentar no docstring que usar o padrão sem um <code>`ProxyFix`</code> correspondente colapsa o limitador num balde único. Opcionalmente, emitir aviso de log no boot quando <code>`default_limits`</code> estiver definido e o app não tiver <code>`ProxyFix`</code> no <code>`wsgi_app`</code> .	SA-02
P3	Elevar o piso de senha ou acrescentar uma verificação contra as senhas mais comuns em <code>`validar_tamanho`</code> — a mesma regra passaria a valer nos quatro aplicativos de uma vez, como já acontece com o piso atual.	SA-03
P3	Manter <code>`sanitizar_log`</code> como está e reforçar no README dos consumidores que ela é rede, não garantia: a defesa primária é não pôr segredo em mensagem. Se algum consumidor precisar de um rótulo novo, ele entra em <code>`CHAVES_SENSIVEIS`</code> , na biblioteca, e não numa cópia local.	SA-05

5. Issues para o GitHub

Cada bloco abaixo é o texto COMPLETO de uma issue, em Markdown, pronto para copiar e colar. Achados triviais do mesmo tema foram agrupados numa issue só, para não gerar ruído no rastreador.

Issue 1 — [Segurança] `resolver\_segredo` aceita qualquer valor não vazio como segredo

```
--- ISSUE 1 ---
Título: [Segurança] `resolver_segredo` aceita qualquer valor não vazio como segredo
Labels: security, severity:baixa, contrato-publico

## Problema

`sharedauth.secrets.resolver_segredo` valida a PROCEDÊNCIA do segredo (o arquivo existe, não está vazio, resolve para `caminho_esperado`) e nunca o CONTEÚDO. Com o padrão `aceitar_variavel=True`, qualquer string não vazia na variável de ambiente é aceita – inclusive um placeholder de documentação, um valor de exemplo ou um segredo reaproveitado da máquina de desenvolvimento.

### Por que é explorável

Não é explorável isoladamente: é a ausência de uma trava. O que a torna relevante é que ela é o ÚNICO ponto em que os quatro consumidores poderiam ser protegidos de uma vez. A forma branda do efeito está nos três irmãos:

- `MegaSena/.env.docker` guarda `POSTGRES_PASSWORD` e `SECRET_KEY` em texto claro;
- `ControleRendaVariavel/.env` guarda `SECRET_KEY`, `DATABASE_URL` (com senha) e `POSTGRES_PASSWORD`;
- `ControleBancario/.env.docker` guarda `POSTGRES_PASSWORD` e `DJANGO_SECRET_KEY`.

Nenhum deles vazou: os arquivos estavam fora do Git e fora da sincronização de nuvem, verificado. E justamente esse o ponto -- nenhuma camada RECUSA esses valores como entrada, porque não existe camada que olhe valor. O que protegeu foi a configuração de ignore de cada projeto, não o contrato.

## Evidência

`sharedauth/secrets.py:93-141`

```python
def resolver_segredo(
 nome: str,
 *,
 ambiente: Mapping[str, str] | None = None,
 aceitar_variavel: bool = True,
 caminho_esperado: Path | None = None,
 obrigatorio: bool = False,
) -> str | None:
 ...
 if aceitar_variavel:
 direto = valores.get(nome)
 if direto:
 return direto
 ...

Impacto

Um segredo de desenvolvimento vira configuração operacional sem que nada acuse. Como a mesma função atende os quatro aplicativos, a lacuna é sistêmica: cada um teria de escrever a validação por conta própria – exatamente o padrão que este pacote existe para eliminar. Hoje o que evita o desfecho ruim é a disciplina de cada projeto com `.gitignore` e `.dropboxignore`, não uma garantia do contrato.

Sugestão de correção

Acrescentar recusa de valor ao contrato, sem mudar o comportamento de quem não pedir:

```python
def resolver_segredo(
    nome: str,
    *,
    ...,
    tamanho_minimo: int | None = None,
    valores_recusados: frozenset[str] = frozenset(),
) -> str | None:
    ...

- `valores_recusados` compara em minúsculas e sem espaço nas pontas, e levanta `SegredoInvalidoError` nomeando a variável – **nunca o valor** (a regra atual do módulo, que deve continuar valendo);
- `tamanho_minimo` cobre o caso mais comum sem exigir lista;
- ambos valem também para o segredo lido por arquivo, não só para a forma direta.
```

Depois de publicar a versão, abrir a issue correspondente em cada um dos quatro consumidores para passarem a informar os parâmetros.

Critérios de aceite

- [] `resolver\_segredo` aceita `valores\_recusados` e `tamanho\_minimo`, ambos opcionais e desligados por padrão
 - [] A recusa levanta `SegredoInvalidoError` e a mensagem NÃO contém o valor recusado
 - [] A validação vale tanto para `NOME\_FILE` quanto para `NOME`
 - [] Teste: valor na lista de recusados levanta; valor equivalente com outra caixa e espaço nas pontas também levanta
 - [] Teste: valor abaixo de `tamanho\_minimo` levanta
 - [] Teste: a mensagem da exceção não contém nenhum trecho do valor recusado
 - [] Chamada sem os novos parâmetros mantém exatamente o comportamento atual (teste de regressão)
 - [] Tag nova publicada e o CHANGELOG/README registrando o contrato novo
- FIM ISSUE 1 ---

Issue 2 — [Segurança] HSTS ausente do conjunto defensivo compartilhado

--- ISSUE 2 ---

Título: [Segurança] HSTS ausente do conjunto defensivo compartilhado

Labels: security, severity:baixa, cabecalhos

Problema

`SECURITY\_HEADERS` traz seis cabeçalhos defensivos e nenhum `Strict-Transport-Security`. `registrar\_cabecalhos` também não o acrescenta. Nos três consumidores Flask, o HSTS existe apenas na camada do Nginx.

Por que é explorável

Um consumidor publicado por HTTPS sem esse proxy à frente – ou com o proxy reconfigurado sem a diretiva – fica sem HSTS por completo, e nada na aplicação acusa a ausência. A primeira requisição de um navegador que ainda não visitou o site pode então ser feita em HTTP claro e interceptada antes do redirecionamento (SSL-strip). Hoje, o que impede o cookie de sessão de viajar nesse cenário é apenas `SESSION\_COOKIE\_SECURE` – uma segunda flag, que pode estar errada de forma independente.

Evidência

```
`sharedauth/security.py:41-50`
```python
SECURITY_HEADERS: dict[str, str] = {
 "X-Content-Type-Options": "nosniff",
 "X-Frame-Options": "DENY",
 "Referrer-Policy": "same-origin",
 "Permissions-Policy": (
 "camera=(), microphone=(), geolocation=(), browsing-topics=()"
),
 "Cross-Origin-Opener-Policy": "same-origin",
 "X-Permitted-Cross-Domain-Policies": "none",
}
```
```

Onde o HSTS de fato existe hoje (fora da biblioteca):

```
`MegaSena/deploy/nginx/megasena.conf:26`
```
add_header Strict-Transport-Security "max-age=31536000" always;
```
```

Impacto

A garantia de transporte depende de configuração fora do repositório da aplicação, replicada por implantação, sem teste que a cubra. É a classe de controle que some numa recriação de servidor sem ninguém notar.

Sugestão de correção

Emitir HSTS a partir da biblioteca, atrás de um parâmetro explícito – nunca incondicional, porque numa instalação local servida por HTTP o cabeçalho trancaria o navegador num esquema que ninguém atende:

```
```python
def registrar_cabecalhos(
 alvo: Flask | Blueprint,
 *,
 imagens_data_uri: bool = False,
 hsts_segundos: int | None = None,
 hsts_incluir_subdominios: bool = True,
) -> None:
```
```

```
O consumidor alimenta `hsts_segundos` com a MESMA flag que já usa para decidir
`https_obrigatorio` em `configurar_sessao` — assim as duas decisões não podem
divergir. `setdefault` mantém a precedência de quem já define o cabeçalho.

## Critérios de aceite

- [ ] `registrar_cabecalhos` aceita `hsts_segundos` (padrão `None` = não emite)
- [ ] Com `hsts_segundos=31536000`, a resposta traz `Strict-Transport-Security:
max-age=31536000; includeSubDomains`
- [ ] Com o padrão, nenhuma resposta ganha o cabeçalho (regressão para instalação local em HTTP)
- [ ] O valor emitido respeita `setdefault`: um consumidor que já defina o cabeçalho continua
vencendo
- [ ] Teste cobrindo os três casos acima
- [ ] README documenta que o parâmetro deve ser alimentado pela mesma flag de
`configurar_sessao(https_obrigatorio=...)`
--- FIM ISSUE 2 ---
```

Issue 3 — [Segurança] Limitador chaveia por REMOTE_ADDR sem forma de o consumidor ajustar

```
--- ISSUE 3 ---
Título: [Segurança] Limitador chaveia por REMOTE_ADDR sem forma de o consumidor ajustar
Labels: security, severity:baixa, rate-limit

## Problema

`iniciar_limiter` fixa `key_func=get_remote_address`. Atrás de um proxy reverso
sem `ProxyFix` — ou com um servidor WSGI que apague os cabeçalhos
`X-Forwarded-*`, como o waitress faz por padrão — `REMOTE_ADDR` é o endereço do
gateway, igual para todas as requisições.

### Por que é explorável

Com a chave colapsada, `default_limits` deixa de ser "por cliente" e vira um
balde único somando o mundo inteiro: quem tenta força bruta divide o orçamento
com os usuários legítimos, e num aplicativo que faz polling isso é autobloqueio
esperando a hora. Não é hipótese — foi o comportamento real do ConfortoTermico,
descrito em `app_factory.py:309-336` daquele repositório, onde o balde único
somava todos os clientes e ainda envenenava o IP registrado na auditoria.

## Evidência

`sharedauth/ratelimit.py:55-62`

```python
limiter = Limiter(
 key_func=get_remote_address,
 default_limits=list(limites_padrao) if limites_padrao is not None else None,
 storage_uri=storage_uri,
 strategy=estrategia,
 enabled=habilitado,
)
```

## Impacto

Onde o consumidor esquecer o `ProxyFix`, a proteção de força bruta do login
existe no código e não existe em produção, sem sintoma até alguém ir medir. Os
quatro consumidores atuais resolvem corretamente por conta própria, mas a
biblioteca não tem como saber disso nem como avisar quando alguém esquecer.

## Sugestão de correção

Duas mudanças pequenas e independentes:

1. aceitar `key_func` opcional em `iniciar_limiter`, mantendo
`get_remote_address` como padrão;
2. no boot, quando `limites_padrao` estiver definido e `app.wsgi_app` não for um
`ProxyFix`, emitir um `logger.warning` dizendo que o limitador pode estar
chaveando pelo IP do proxy. Aviso, não erro: há instalações legítimas sem
proxy nenhum.

## Critérios de aceite

- [ ] `iniciar_limiter` aceita `key_func`, com `get_remote_address` como padrão
- [ ] Teste: `key_func` própria é de fato usada pelo `Limiter` construído
- [ ] Teste: chamada sem `key_func` mantém o comportamento atual
- [ ] O aviso de boot aparece quando há `default_limits` e não há `ProxyFix`, e não aparece nos
demais casos
- [ ] O docstring explica a armadilha do balde único, com o caso do ConfortoTermico como exemplo
--- FIM ISSUE 3 ---
```

Issue 4 — [Segurança] Política de senha limitada ao piso de 8 caracteres

```

--- ISSUE 4 ---
Título: [Segurança] Política de senha limitada ao piso de 8 caracteres
Labels: security, severity:informativa, senhas

## Problema

A política de senha da biblioteca é uma regra só: `len(senha) >= 8`. Não há
verificação contra senha comum, contra semelhança com o nome de usuário nem
contra lista de senhas vazadas. Como `gerar_hash` é o caminho único de gravação
nos três consumidores Flask, essa é a política efetiva deles.

### Por que é explorável

`senha123` e `12345678` passam. Com o limite padrão de 10 tentativas por minuto
por IP, um adivinhador com lista curta ainda dispõe de várias centenas de
tentativas por hora contra uma conta cujo nome ele conheça.

## Evidência

`sharedauth/passwords.py:26` e `sharedauth/passwords.py:52-56`

```python
MIN_PASSWORD_LENGTH = 8

def validar_tamanho(senha: str) -> None:
 """Levanta :class:`SenhaMuitoCurtaError` se a senha for curta demais."""
 if len(senha) < MIN_PASSWORD_LENGTH:
 raise SenhaMuitoCurtaError(
 f"A senha deve ter pelo menos {MIN_PASSWORD_LENGTH} caracteres."
)
 ...

Impacto

Baixo no modelo de uso atual: poucas contas, todas provisionadas por um
administrador, e a senha inicial é sempre a temporária gerada por
`gerar_senha_temporaria` (12 caracteres, alfabeto de 58, ~70 bits, sorteados com
`secrets.choice`). O risco fica na senha que o dono escolhe DEPOIS da troca
obrigatória – que é justamente a que vale por mais tempo.

Sugestão de correção

Acrescentar a `validar_tamanho` (ou a uma `validar_politica` nova que a
substitua nos pontos de gravação) uma checagem contra uma lista curta e
embarcada de senhas comuns, e opcionalmente contra o nome de usuário quando o
consumidor puder informá-lo. Manter o piso de 8 ou elevá-lo é decisão do
mantenedor, mas deve ser uma decisão explícita e registrada, não o padrão
herdado.

O ControleBancario já resolve isso por conta própria com o
`CommonPasswordValidator` do Django – vale como referência do comportamento
esperado.

Critérios de aceite

- [] Existe verificação contra senha comum no caminho de gravação de senha
- [] A exceção continua sendo um `ValueError` (para não quebrar quem já captura a família)
- [] A mensagem de recusa não revela qual entrada da lista casou
- [] Teste: `senha123` e `12345678` são recusadas; uma senha forte de 8 caracteres continua
 aceita
- [] A decisão sobre manter ou elevar `MIN_PASSWORD_LENGTH` fica registrada no README ou num
 ADR
--- FIM ISSUE 4 ---

```